Start coding or generate with AI.

Lab task

```
import heapq

# A* Algorithm
def a_star(graph, start, goal, heuristic):
    open_list = []
    heapq.heappush(open_list, (0, start))

    g_cost = {node: float('inf') for node in graph}
    g_cost[start] = 0

    came_from = {}

    while open_list:
        current_f, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        for neighbor, cost in graph.get(current, []):  # direc
            tentative_g = g_cost[current] + cost

            if tentative_g < g_cost.get(neighbor, float('inf')
                came_from[neighbor] = current
                g_cost[neighbor] = tentative_g

                f_cost = tentative_g + heuristic(neighbor, goa
                heapq.heappush(open_list, (f_cost, neighbor))

    return None


# Directed graph
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 1)],
    'E': [('G', 2)],
    'F': [('G', 5)],
```

```
        'G': []
    }

    # Heuristic function
    def heuristic(node, goal):
        h_values = {
            'A': 7,
            'B': 6,
            'C': 5,
            'D': 2,
            'E': 1,
            'F': 3,
            'G': 0
        }
        return h_values[node]


    # Run A*
    start = 'A'
    goal = 'G'

    path = a_star(graph, start, goal, heuristic)

    print("Optimal Path:", path)
```

```
Optimal Path: ['A', 'B', 'D', 'G']
```